

A blue background with a grid of white line-art icons. The icons include a document, a tag, a puzzle piece, a magnifying glass, a smartphone, a document with lines, a tag, a puzzle piece, a magnifying glass, a smartphone, a document with lines, an envelope, a speech bubble, a target with an arrow, two interlocking gears, a pie chart, an envelope, a speech bubble, a target with an arrow, two interlocking gears, a pie chart, a checkmark in a circle, a presentation board with a line graph, a thumbs up, a lightbulb, a clock, a checkmark in a circle, a presentation board with a line graph, a thumbs up, a lightbulb, a clock, and a checkmark in a circle.

BUILDING APPLICATIONS WITH PYTHON AND WEB FRAMEWORKS

2

Prerequisites

PYTHON FOR EVERYBODY

You can read Chapters 2,3,4,8,9,10,11 from

http://do1.dr-chuck.com/pythonlearn/EN_us/pythonlearn.pdf

We could also watch the chapters from this book in video lectures from <https://www.py4e.com/lessons>

The **Web Server Gateway Interface** (WSGI) is a specification for simple and universal interface between web servers and web applications or frameworks for the Python programming language.

HTML

We will become familiar with simple HTML and understand forms and HTTP methods (post and get).

MVC

We will learn the basic concepts of web frameworks and MVC patterns.

3

Python and Database interaction

4

Tools

- ▶ Python
- ▶ Bottle
- ▶ PyMySQL

5

Architecture

- ▶ Browser (HTML Form)
- ▶ HTTP Request (GET/POST)
- ▶ Bottle Route (Python)
- ▶ SQL Query μέσω PyMySQL
- ▶ MySQL Database
- ▶ HTML Response

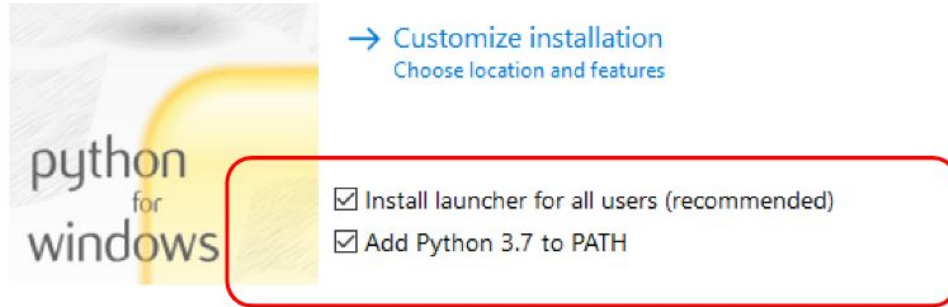
6

For windows systems just visit <https://www.python.org/downloads/> and select the latest version.



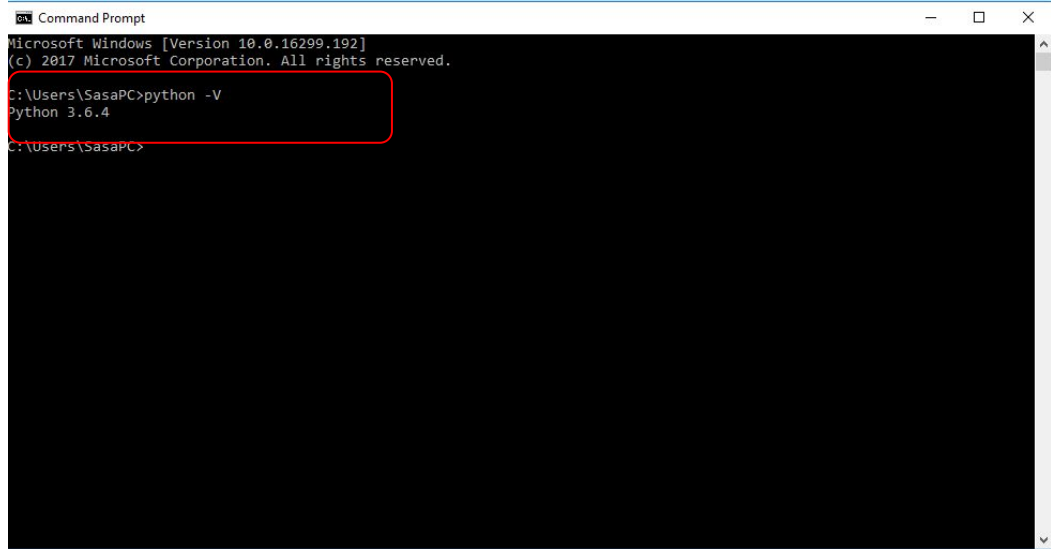
7

Enable the “**Add Python 3.x.x to PATH**” .



8

Open a command prompt window and check your installation.



```
Command Prompt
Microsoft Windows [Version 10.0.16299.192]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\Users\SasaPC>python -V
Python 3.6.4

C:\Users\SasaPC>
```

9

Bottle and pymysql

- ▶ Install PyMySQL ,
<https://pypi.org/project/PyMySQL/>
- ▶ Install Bottle
<https://bottlepy.org/docs/dev/>
- ▶

10

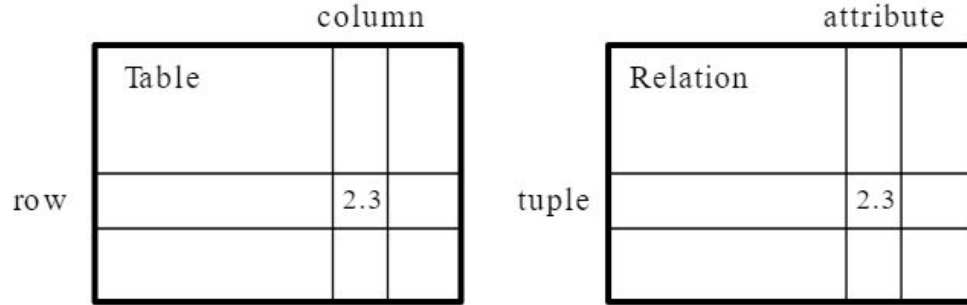
Python and Database manipulation

- ▶ We will next focus on using Python to work with data in MySQL.

11

Database Concepts

The primary data structures in a database are: tables, rows, and columns.



In technical descriptions of relational databases the concepts of table, row, and column are more formally referred to as ***relation***, ***tuple***, and ***attribute***, respectively.

12

Python and Database manipulation

Before connecting to a MySQL database, make sure you have created a database TESTDB.

- We will create a table EMPLOYEE in TESTDB.
- This table has fields FIRST_NAME, LAST_NAME, AGE, SEX and INCOME.
- User ID and password are set to access TESTDB.
- Python module PyMySQL should be installed properly on your machine.
- You understand MySQL Basics ;-).

13

Python and Database manipulation

Example Python MySQL connector

```
import pymysql

# Open database connection
db = pymysql.connect("localhost","myuser","mypassword","TESTDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

# execute SQL query using execute() method.
cursor.execute("SELECT VERSION()")

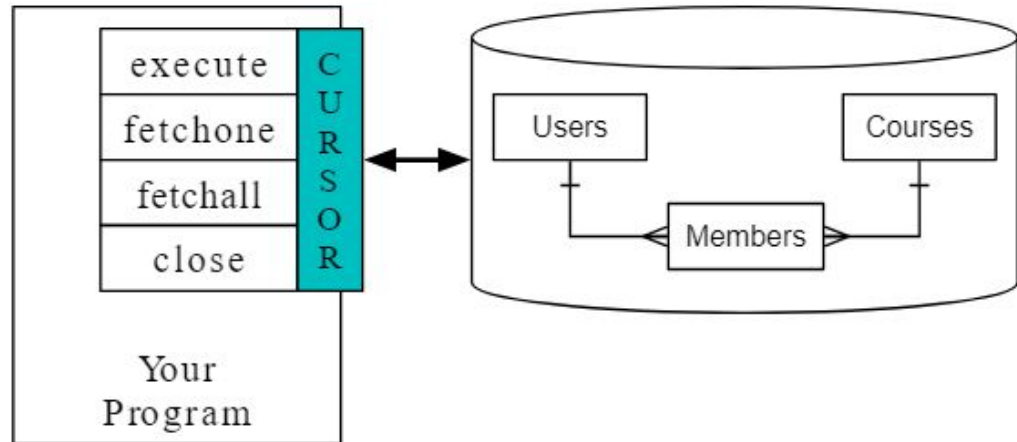
# Fetch a single row using fetchone() method.
data = cursor.fetchone()
print ("Database version : %s " % data)

# disconnect the Database connection
db.close()
```

14

Python and Database manipulation

- ▶ A **cursor** is like a file handle that we can use to perform operations on the data stored in the database. Calling ***cursor()*** is very similar conceptually to calling *open()* when dealing with text files.
 - If a connection is established with the datasource, then a Connection Object is returned and saved into **db** for further use, otherwise **db** is set to None.



15

Python and Database manipulation

- ▶ Once we have the cursor, we can begin to execute commands on the contents of the database using the ***execute()*** method.
 - Execute returns **an iterator** over the results.
 - Rows are returned as **python tuples**

16

Python iterators

An iterator is an object that contains a countable number of values.

Technically, in Python, an iterator is an object which implements the iterator protocol, which consist of the methods `__iter__()` and `__next__()`.

Example

Return an iterator from a tuple, and print each value:

```
mytuple = ("apple", "banana", "cherry")  
myit = iter(mytuple)  
  
print(next(myit))  
print(next(myit))  
print(next(myit))
```

17

Python iterators

Example

Iterate the values of a tuple:

```
mytuple = ("apple", "banana", "cherry")
```

```
for x in mytuple:  
    print(x)
```

You can read more on iterators here:

<https://anandology.com/python-practice-book/iterators.html>

18

Συνδεση με βαση (ΥΠΟΧΡΕΩΤΙΚΑ)

```
import pymysql
```

```
def get_connection():
```

```
    return pymysql.connect(
```

```
        host="localhost",
```

```
        user="root",
```

```
        password="password",
```

```
        database="airbnb",
```

```
        cursorclass=pymysql.cursors.DictCursor
```

```
    )
```

19

Python and Database manipulation

Let's create the database Table using Python

```
import pymysql

# Open database connection
db = get_connection()

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Drop table if it already exist using execute() method.
cursor.execute("DROP TABLE IF EXISTS EMPLOYEE")

# Create table as per requirement
sql = """CREATE TABLE EMPLOYEE (
    FIRST_NAME  CHAR(20) NOT NULL,
    LAST_NAME   CHAR(20),
    AGE INT,
    SEX CHAR(1),
    INCOME FLOAT )"""

cursor.execute(sql)

# disconnect from server
db.close()
```

20

Python and Database manipulation

An INSERT operation

```
import pymysql

# Open database connection
db = get_connection()

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Prepare SQL query to INSERT a record into the database.
sql = """INSERT INTO EMPLOYEE(FIRST_NAME,
    LAST_NAME, AGE, SEX, INCOME)
    VALUES ('Mac', 'Mohan', 20, 'M', 2000)"""
try:
    # Execute the SQL command
    cursor.execute(sql)
    # Commit your changes in the database
    db.commit()
except:
    # Rollback in case there is any error
    db.rollback()

# disconnect from server
db.close()
```

21

Python and Database manipulation

READ Operation

READ Operation on any database means to fetch some useful information from the database.

- ▶ **fetchone()** – It fetches the next row of a query result set. A result set is an object that is returned when a cursor object is used to query a table.
- ▶ **fetchall()** – It fetches all the rows in a result set. If some rows have already been extracted from the result set, then it retrieves the remaining rows from the result set.
- ▶ **rowcount** – This is a read-only attribute and returns the number of rows that were affected by an execute() method.

22

Python and Database manipulation

```
import pymysql

# Open database connection
db = get_connection()

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Prepare SQL query to INSERT a record into the database.
sql = "SELECT * FROM EMPLOYEE WHERE INCOME > '%d'" % (1000)
try:
    # Execute the SQL command
    cursor.execute(sql)
    # Fetch all the rows in a list of lists.
    results = cursor.fetchall()
    for row in results:
        fname = row[0]
        lname = row[1]
        age = row[2]
        sex = row[3]
        income = row[4]
        # Now print fetched result
        print ("fname = %s,lname = %s,age = %d,sex = %s,income = %d" % \
              (fname, lname, age, sex, income ))
except:
    print ("Error: unable to fetch data")

# disconnect from server
db.close()
```

23

Python and Database manipulation

UPDATE Operation on any database means to update one or more records, which are already available in the database.

```
#!/usr/bin/python3

import pymysql

# Open database connection
db = get_connection()

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Prepare SQL query to UPDATE required records
sql = "UPDATE EMPLOYEE SET AGE = AGE + 1
      WHERE SEX = '%c'" % ('M')

try:
    # Execute the SQL command
    cursor.execute(sql)
except:
    # Rollback in case there is any error
    db.rollback()

try:    # Commit your changes in the database
    db.commit()
except:
    db.rollback()

# disconnect from server
db.close()
```

In general, try to make your try blocks as short as logically possible

Uses two methods to either commit or rollback a transaction.

24

Python and Database manipulation

Transactions are a mechanism that ensures data consistency. Transactions have the following four properties

Atomicity – Either a transaction completes or nothing happens at all.

Consistency – A transaction must start in a consistent state and leave the system in a consistent state.

Isolation – Intermediate results of a transaction are not visible outside the current transaction.

Durability – Once a transaction was committed, the effects are persistent, even after a system failure.

A decorative header featuring a large blue number '25' on the left. To its right is a horizontal band filled with various light blue line-art icons. These icons include a document, a tag, a puzzle piece, a magnifying glass, a smartphone, a document with lines, a target with an arrow, a gear, a pie chart, an envelope, a speech bubble, a thumbs up, a lightbulb, a clock, a checkmark, and a presentation board with a line graph.

25

Bottle web framework

26

- ▶ Bottle is micro web-framework
- ▶ A Web framework is a set of **components** designed to simplify your web development process.
- ▶ It has basic **structuring** tools in it, which serve as a solid base for your project.
- ▶ It allows you to focus on the most important details and project's goals instead of creating things, that you can simply pull out of the framework.

27

Hello example

- ▶ Let's start with a very basic "Hello World" example:

```
from bottle import route, run

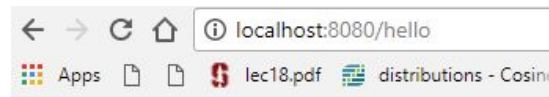
@route('/hello')
def hello():
    return "Hello World!"

run(host='localhost', port=8080, debug=True)
```

- ▶ Write this script in a text editor of your choice, name it **mybottletutor.py** and run this script on command prompt

```
C:\Users\SasaPC\Documents\DB_Lab\MYTUTOR>python mybottletutor.py
Bottle v0.13-dev server starting up (using WSGIRefServer())...
Listening on http://localhost:8080/
Hit Ctrl-C to quit.
```

- ▶ Visit ***http://localhost:8080/hello*** from your web browser and you will see "Hello World!".



Hello World!

28

How it works

- ▶ The **route()** decorator binds a piece of code to an URL path. In this case, we link the /hello path to the hello() function.
- ▶ This is called a route and is the most important concept of this framework . ***Routing is the process of mapping a requested URL to the code responsible for generating the associated HTML.***
- ▶ You can define as many routes as you want.
- ▶ Whenever a browser requests an URL, the associated function is called and the return value is sent back to the browser.
- ▶ Basically, it means if you go to your website with a '/hello' at the end, the code in the hello() function will be run, and whatever is returned will be shown in your browser.

```
from bottle import route, run

@route('/hello')
def hello():
    return "Hello World!"

run(host='localhost', port=8080, debug=True)
```

29

ROUTING STATIC FILES

Static files such as images, CSS or html files are not served automatically. You have to add a route and a callback, to control which files get served and where to find them:

```
from bottle import static_file
@route('/static/<filename>')
def server_static(filename):
    return static_file(filename,
        root='/path/to/your/static/files')
```

The **static_file()** function is a helper to serve files in a safe and convenient way (see Static Files).

This example is limited to files directly within the `/path/to/your/static/files` directory because the `<filename>` wildcard won't match a path with a slash in it.

30

Forms and HTTP Requests

- ▶ In the following example we present a simple form and we display a simple message (**no form processing is performed in the example**)

```
from bottle import run, post, route
```

```
@route('/login')
```

```
def login():
```

```
    return '''
```

```
        <form action="/login" method="post">
```

```
            Username: <input name="username" type="text" />
```

```
            Password: <input name="password" type="password" />
```

```
            <input value="Login" type="submit" />
```

```
        </form>
```

```
    '''
```

```
@route('/login', method='POST')
```

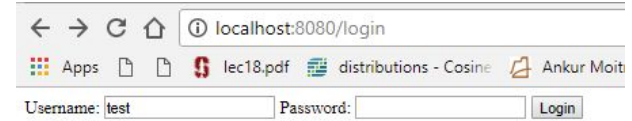
```
def do_login():
```

```
    username = request.forms.get('username')
```

```
    password = request.forms.get('password')
```

```
    return "<p>your login information was correct! </p>"
```

```
run(host='localhost', port=8080, debug=True)
```



31

Make pages with HTML

So now we can send data to the Python code, and the Python code can compute using that data and return data back to the browser.

```
# using HTML to make it look nicer
```

```
@route('/display_list')
def display_raw_html():
    my_list = [1,2 ,3 4, 5, 6, 7, 8 , 9]
    # First we put the stuff at the top, adding "n" in there
    html = "<h1>The "+len(factors_list)+" list items
are</h1>"+ "\n" + "<ul>"

    # for each factor, we make a <li> item for it
    for f in factors_list:
        html += "<li>"+str(f)+"</li>"+ "\n"
    html += "</ul> </body>" # the close tags at the bottom
    return html
```

32

Templates

A **template** in Bottle is an HTML file with embedded Python-like placeholders that gets filled with data on the server before being sent to the browser.

Instead of returning plain text like:

```
return "yes"
```

You return a **rendered** HTML page where variables are inserted.

33

SELECT clause example

```
@route('/check_property', method='POST')
def check_property():
    location = request.forms.get('location')
    property_type = request.forms.get('property_type')

    conn = get_connection()
    with conn.cursor() as cur:
        sql = """
        SELECT COUNT(*) > 0 AS exists_flag
        FROM Property
        WHERE location = %s AND property_type = %s
        """

        cur.execute(sql, (location, property_type))
        result = cur.fetchone()
    conn.close()

    answer = "yes" if result["exists_flag"] else "no"
    return template('result', answer=answer)
```

34

Why Use Templates?

Without templates:

- ▶ Backend returns raw strings or JSON
- ▶ No structured page
- ▶ No separation of logic and UI

With templates:

- ▶ Backend handles logic (DB, decisions)
- ▶ Template handles presentation (HTML)
- ▶ Same page layout reused with different data

35

What is a .tpl File

A **.tpl** file is just HTML with special syntax.

```
<!DOCTYPE html>
<html>
<head>
  <title>Property Check</title>
</head>
<body>
  % if answer == "yes":
    <h2 style="color:green;">Property exists </h2>
  % else:
    <h2 style="color:red;">Property does not exist </h2>
  % end
</body>
</html>
```

36

Template Syntax

{{variable}}

% if, %

% end

Print a variable
for Python logic
Close block

37

What Happens in Your Route

```
@route('/check_property', method='POST')  
def check_property():
```

This runs when a form sends data to
/check_property.

You get form data:

```
location = request.forms.get('location')  
property_type =  
request.forms.get('property_type')
```

38

Then query DB:

```
SELECT COUNT(*) > 0 AS exists_flag
```

This returns **True** or **False**.

39

Data flow

The Key Line: `return template()`

```
answer = "yes" if result["exists_flag"] else "no"  
return template('result', answer=answer)
```

This does 3 things:

1. Finds the template file (Bottle looks for: `views/result.tpl`)
2. Passes variables to template (You send: `answer = "yes"` or `"no"` and Inside template it becomes available as: `{{answer}}`)
3. Renders final HTML (Bottle replaces template logic → creates full HTML page → sends to browser.)

ΚΑΝΟΝΕΣ ΓΙΑ ΤΗΝ ΕΡΓΑΣΙΑ

41

RULES!!!

Κανόνας ΑΣΦΑΛΕΙΑΣ

ΑΠΑΓΟΡΕΥΕΤΑΙ string formatting σε SQL

ΛΑΘΟΣ

```
sql = "SELECT * FROM users WHERE name = '%s'" % name
```

ΣΩΣΤΟ

```
sql = "SELECT * FROM users WHERE name = %s"  
cursor.execute(sql, (name,))
```

42

Rules

Τι ΔΕΝ είναι αποδεκτό

- ▶ SQL μέσα σε HTML
- ▶ Χωρίς commit σε αλλαγές
- ▶ Χωρίς parameterized queries
- ▶ Μόνο static HTML

43

Rules

Τι ΔΕΝ είναι αποδεκτό

Δεν επιτρέπεται καμία εντολή που δεν έχετε διδαχθεί στο μάθημα

Ενδεικτικά παραδείγματα που **δεν επιτρέπονται** είναι τα παρακάτω:

Τα SELECT μέσα στο SELECT και το FROM clause δεν επιτρέπονται.

Το JOIN δεν επιτρέπεται καθόλου ως εντολή (δηλαδή δεν επιτρέπονται LEFT JOIN, CROSS JOIN, INNER JOIN κτλ...).

Δεν επιτρέπεται η χρήση της LIMIT.

4

Project structure

4

Δομή Project (ενδεικτικό)

project/

app.py → (routes, forms, return template)

db.py → (sql functions, connections, sql
queries, commit, fetch)

templates/index.tpl

templates/result.tpl

A close-up photograph of a hand holding a blue pen, poised to write on a piece of paper. The hand is wearing a grey, textured sweater sleeve. The background is blurred, showing a desk and some papers.

45

THANKS!

Any questions?

Post your question at **eclass!**

You can reach me at akolovou@di.uoa.gr